
Cahier de conception

Outil de génération automatique de kits graphiques d'opérations commerciales

Test de développement : alternance Développeur IA

Société **Gefradis** (groupe Im4Pulse)

Auteur : Vaneck

Version : 0.6 (conception)

Date : 22 juin 2026

Document technique décrivant la conception de l'application avant le développement. Le cahier des charges (le besoin) est défini par la note de cadrage fournie par le client ; le présent document décrit la solution retenue.

Table des matières

1	Contexte et objectif	3
1.1	Contexte	3
1.2	Objectif	3
1.3	Périmètre	3
2	Besoins fonctionnels	3
3	Besoins non fonctionnels	4
4	Catalogue des formats à produire	4
5	Spécification graphique (charte)	4
5.1	Couleurs	4
5.2	Typographie, logo, ton	4
6	Architecture	5
6.1	Architecture fonctionnelle (exécution locale)	5
6.2	Pile technique	5
7	Cas d'utilisation	5
8	Vue dynamique : séquence de génération	5
9	Contrat de données	6
9.1	Entrée : le brief (formulaire)	6
9.2	Sortie de Claude : le copy structuré	6
10	Système de design et génération créative	7
10.1	Le montage composable : une disposition par format	8
10.2	La banque d'images	8
10.3	Une offre générique	8
11	Parcours utilisateur (UX)	9
11.1	Style de l'interface	9
12	Rôle de l'IA et stratégie de prompt	10
13	Plan de construction (lotissement)	10
14	Déploiement et mise en production (GCP)	10
14.1	Conteneurisation	11
14.2	Architecture en production	11

14.3 Deux points d'attention résolus	11
15 Décisions, hypothèses et question ouverte	11
15.1 Décisions assumées	12
15.2 Points clarifiés avec le client	12
16 Risques et limites	12
17 Critères de réussite et tests	12
18 Annexe : exemple de prompt	13

1. Contexte et objectif

1.1. Contexte

Gefradis est une entreprise de menuiserie PVC sur-mesure (portes, fenêtres), qui vend en ligne aux particuliers et aux professionnels. Le service marketing mène régulièrement des **opérations commerciales**. Aujourd'hui, la production des visuels (les bannières) de ces opérations dépend de ressources créatives et graphiques, ce qui ralentit le déploiement des campagnes.

1.2. Objectif

Mettre à disposition du métier marketing un **outil web** qui lui permet de réaliser, de manière autonome et sans graphiste, un **kit graphique** complet (un ensemble de bannières à tous les formats utiles), conforme à la charte Gefradis, prêt à être déployé sur le site et sur les plateformes publicitaires (Google et Meta).

1.3. Périmètre

- **Dans le périmètre** : la saisie d'un brief, la génération du texte des bannières par IA, la composition des bannières à la charte (tous les formats de la note), l'aperçu et l'ajustement avant l'export, le téléchargement du kit. Versions statique et animée.
- **Hors périmètre (pour ce rendu)** : la gestion des comptes utilisateurs, la connexion directe aux régies publicitaires (publication automatique) et un éditeur graphique manuel libre.

2. Besoins fonctionnels

L'outil suit le principe **entrée, traitement, sortie**.

- **F1. Saisie du brief** : l'utilisateur décrit la mécanique commerciale (1.1), le message commercial (1.2), en option la direction créative (1.3 : style et angle souhaités), et choisit dans le formulaire la **couleur de fond**, la présence d'une **image produit** et le texte du **bouton** (menus déroulants et champs).
- **F2. Sélection des formats** : l'utilisateur coche les bannières à produire (site, Google Display, Google Pmax, Meta).
- **F3. Génération du texte** : le système transforme le brief en contenu structuré (accroche, offre de type montant, pourcentage ou texte, paliers, date) et déduit la catégorie et, pour une opération transversale, le thème (ex. Black Friday). L'image et le bouton, eux, viennent du formulaire.
- **F4. Composition** : pour chaque format, le système calcule une **disposition** et assemble le texte, les couleurs, la police, le logo et, si demandé, l'image (banque d'images), dans un **gabarit composable unique** où le contenu occupe tout l'espace et est mis à l'échelle, aux dimensions exactes et à la charte.
- **F5. Aperçu** : le système affiche toutes les bannières générées.
- **F6. Ajustement et régénération** : l'utilisateur peut modifier le brief et relancer la génération autant de fois que nécessaire.
- **F7. Téléchargement** : le système livre le kit complet sous la forme d'une archive **.zip**.

3. Besoins non fonctionnels

- **NF1. Conformité à la charte** : respect strict et systématique des couleurs, de la police (Cabin), du logo et du ton de la marque.
- **NF2. Simplicité** : l'outil doit être utilisable par un profil non technique, sans logiciel de graphisme.
- **NF3. Rapidité** : la génération d'un kit s'effectue en un temps raisonnable (de quelques secondes à quelques dizaines de secondes).
- **NF4. Maîtrise des coûts** : usage raisonné de l'API (le modèle ne génère que du texte court).
- **NF5. Sécurité** : la clé d'API n'est jamais écrite dans le code (fichier `.env` exclu du dépôt).
- **NF6. Reproductibilité** : environnement isolé (venv) décrit par un fichier de dépendances.

4. Catalogue des formats à produire

Les dimensions du site ont été communiquées par le client ; celles des régies publicitaires sont des standards.

Destination	Élément	Dimensions (px) / type
Site	Header page d'accueil	Desktop 1900x456, Mobile 426x575
Site	Bandeau page catégorie	2560x280 (unique, sans version mobile)
Google Display	Pavé / Bannière / Skyscraper / Mobile	300x250, 728x90, 160x600, 320x50 (statique et animé)
Google Pmax	Visuels statiques	Paysage 1200x628, Carré 1200x1200, Portrait 960x1200
Meta (FB / Insta)	Carré / Portrait / Story	1080x1080, 1080x1350, 1080x1920

5. Spécification graphique (charte)

Ces éléments constituent les *jetons de design* (design tokens) appliqués par le moteur de composition.

5.1. Couleurs

Nom	Hex	Usage
St Patricks blue	#253081	Fond principal des bannières
Midnight blue	#040C4D	Variante foncée
Cyber yellow	#FFD52B	Accent et mise en avant du chiffre clé
Beige	#FFFDE5	Fond clair / arrière-plan
Blanc	#FFFFFF	Texte sur fond bleu

5.2. Typographie, logo, ton

- **Police** : Cabin (Regular et Bold). Elle est *self-hosted* (fichier `.ttf` embarqué dans le projet via `@font-face`), avec une police sans-serif système en repli. Pas de dépendance à un CDN au moment du rendu.

- **Logo** : deux variantes (bleu pour fond clair ; badge bleu/jaune pour fond foncé), posées selon la couleur du fond, et **uniquement sur les bannières publicitaires** (Google, Meta) ; pas de logo sur les bannières site (on est déjà dans l'environnement Gefradis). Les fichiers fournis ayant un fond blanc, une version transparente est générée automatiquement.
- **Ton** : professeur ou mentor, précis, professionnel, clair sans être trop technique.
- **Imagerie** : esthétique épurée, minimaliste, « sortie d'usine ».

6. Architecture

6.1. Architecture fonctionnelle (exécution locale)

La figure 1 montre les composants et le flux principal.

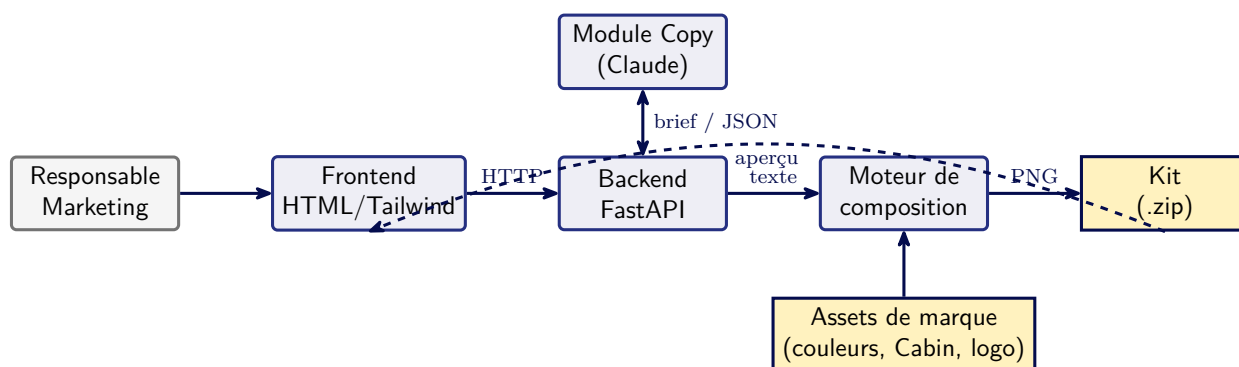


FIGURE 1 – Architecture fonctionnelle (exécution locale).

6.2. Pile technique

Couche	Technologie
Langage	Python
Backend / API	FastAPI
Frontend (interface)	HTML/CSS et Tailwind
Génération de texte	API Claude (SDK <code>anthropic</code>)
Composition d'images	HTML/CSS rendu en PNG via Playwright
Animations	GIF, HTML5, MP4 (à partir d'une source unique)
Production	Cloud Run, Artifact Registry, Secret Manager, Cloud Storage

7. Cas d'utilisation

La figure 2 présente le diagramme de cas d'utilisation. L'acteur unique est le **Responsable Marketing**.

Scénario nominal (*Générer le kit*) : l'utilisateur saisit le brief et sélectionne les formats, puis déclenche la génération ; le système produit les bannières et les affiche ; l'utilisateur prévisualise, ajuste si besoin (la boucle), puis télécharge le kit.

8. Vue dynamique : séquence de génération

La figure 3 détaille les échanges lors d'une génération de kit, y compris la **visualisation de l'aperçu** et la **boucle de régénération** : l'utilisateur ajuste son brief et relance autant de fois

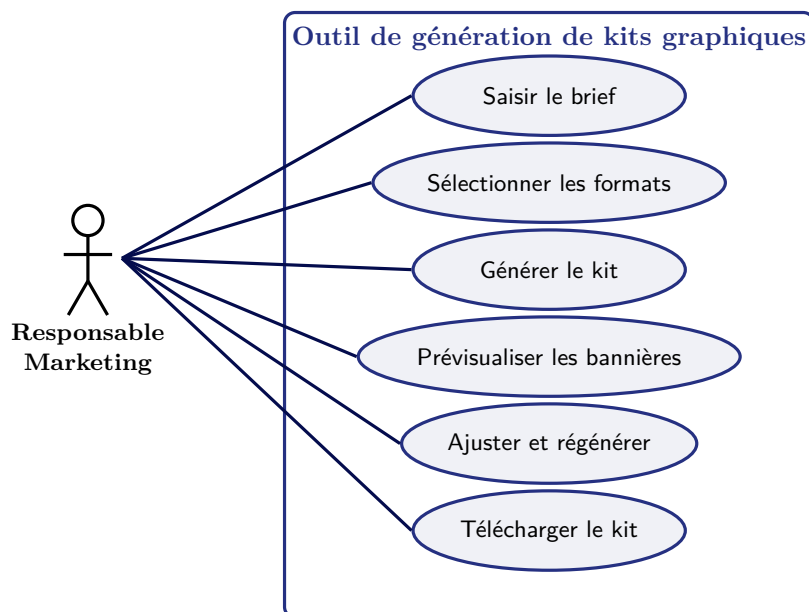


FIGURE 2 – Diagramme de cas d'utilisation.

que nécessaire (cadre *loop*) avant de télécharger le kit.

9. Contrat de données

Le *contrat de données* définit les structures échangées entre l'interface, l'IA et le moteur.

9.1. Entrée : le brief (formulaire)

- **mecanique** (1.1) : texte libre (conditions et niveaux de remise).
- **message** (1.2) : nom de l'opération et accroche.
- **illustrations** (1.3) : direction créative, style et angle souhaités (optionnel). À défaut, la charte s'applique.
- **illustrer** : avec ou sans **image produit** (menu déroulant).
- **cta** : texte du **bouton** d'appel à l'action (champ ; vide = pas de bouton).
- **couleur_fond** : couleur de fond choisie dans la palette (menu déroulant ; #253081 par défaut).
- **formats** : liste des bannières à produire (cases cochées).

9.2. Sortie de Claude : le copy structuré

Claude renvoie un **JSON** (le contrat entre l'IA et le moteur). Exemple pour le cas de test :

```

{
  "categorie": "transversale",
  "titre": "Gros projets = Grosses Remises",
  "offre_type": "montant",
  "offre_label": "Jusqu'à",
  "offre_nombre": "500",
  "offre_suffixe": "EUR offerts",
  "offre_texte": "",
  "paliers": [{
    "condition": "Des 1000 EUR HT",
    "valeur": "100 EUR offerts"
  }, {
    "condition": "Des 3000 EUR HT",
    "valeur": "350 EUR offerts"
  }],
  "date_validite": "Jusqu'au 30 juin 2026",
  "theme_transverse": ""
}
  
```

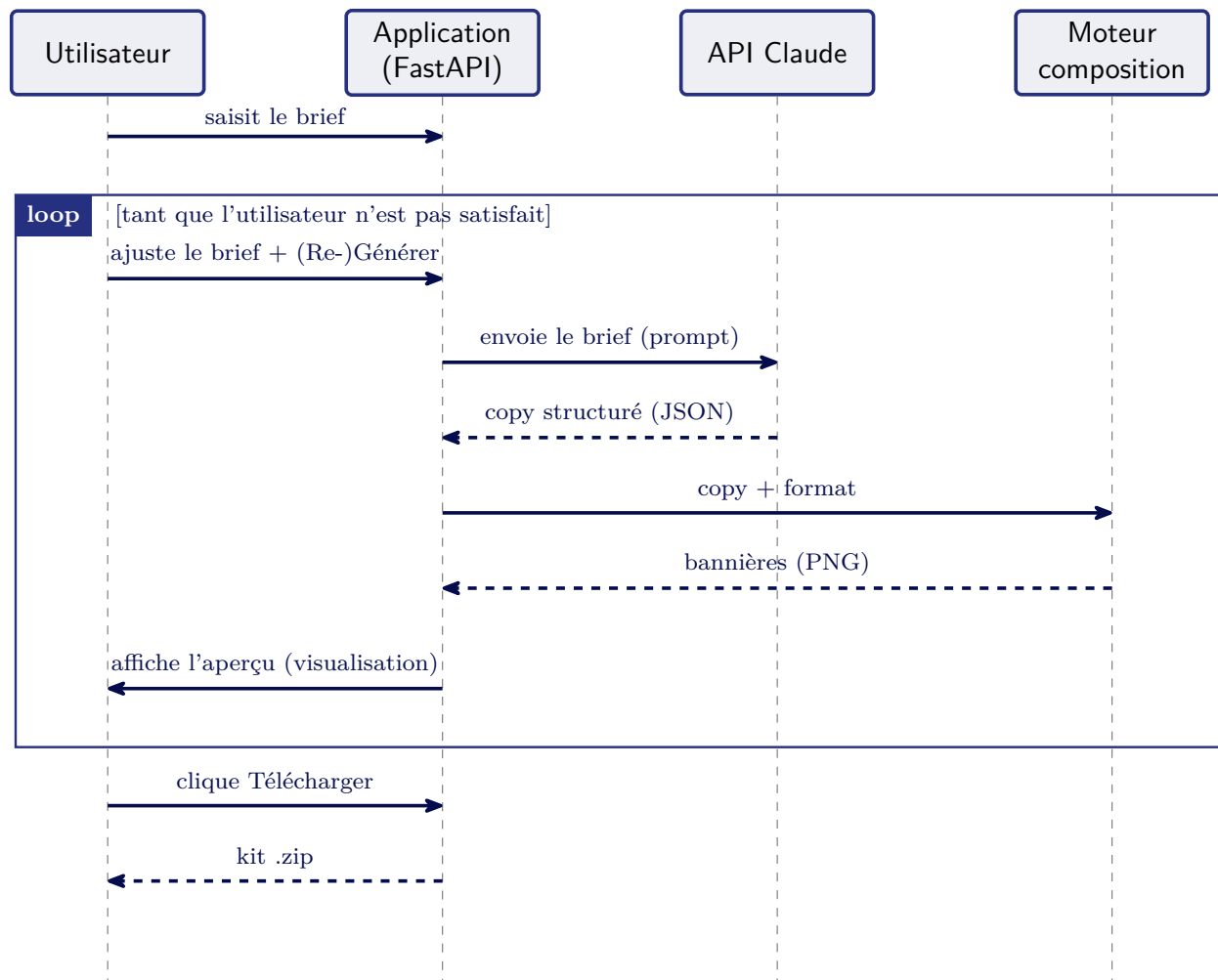


FIGURE 3 – Diagramme de séquence : génération, visualisation de l'aperçu et boucle de régénération.

```

}

```

Claude ne produit que ce JSON, jamais de HTML. Il ne rédige que le **texte** (accroche, offre, paliers, date) et déduit quelques paramètres (catégorie, type d'offre, thème transverse). L'**image**, le **bouton** et la **couleur de fond** ne viennent PAS de Claude : ce sont des champs du **formulaire**. C'est ensuite le code Python qui injecte ces valeurs dans le gabarit, avant le rendu par Playwright ; ce choix garantit l'absence d'erreurs de parsing (HTML tronqué, guillemets mal échappés) et le respect strict des dimensions. Les champs clés : **categorie** (une des familles produit, ou **transversale**) sert à choisir l'image ; **offre_type** (**montant**, **pourcentage** ou **texte**) commande le rendu de l'offre ; **paliers** porte les niveaux de remise (2 à 3) pour une mécanique par paliers ; **date_validite** l'échéance en mention complète ; **theme_transverse** sélectionne le visuel d'une opération transversale (ex. Black Friday). Le **même message** est conservé sur tous les formats : le gabarit adapte la taille du contenu à chaque format.

10. Système de design et génération créative

L'outil est **générique** : il ne reproduit pas un modèle unique, il produit des bannières **variées**, adaptées à chaque brief et à chaque format. La charte n'est pas un moule, mais un **cadre** à respecter. La conception repose sur deux rôles complémentaires.

- **Le système de design (les contraintes)** : la charte encodée dans un **gabarit composable unique**. Le code calcule, pour chaque format, une **disposition** et y place les briques de la charte (logo, accroche, offre ou paliers, bouton, image) avec **toute la palette**, la police Cabin et le logo. Le contenu **occupe tout l'espace disponible** et est **mis à l'échelle** pour tenir exactement dans le format, sans jamais déborder. Chaque rendu est conforme à la charte : c'est le cadre non négociable qui garantit le respect de la marque et des dimensions.
- **L'IA, concepteur-rédacteur (la créativité)** : à partir du brief, Claude rédige le copy (accroche, offre, paliers, date) et **déduit** la catégorie, le type d'offre et, pour une opération transversale, le thème. L'**image**, le **bouton** et la **couleur de fond** sont choisis dans le **formulaire** (plus prévisible). Claude ne génère pas de HTML : c'est le code qui rend.

Ce choix (l'IA *rédige*, le code *compose et rend*) est le plus **fiable** : aucune erreur de HTML généré, des dimensions toujours exactes, la charte toujours respectée. La créativité vient du copy et du **montage** (disposition adaptée au format, occupation de l'espace), pendant que le cadre visuel reste verrouillé.

Style par défaut : le menu de couleur de fond propose le **bleu St Patricks #253081** par défaut ; sans image (option du formulaire), on obtient une variante purement **typographique**, très propre.

10.1. Le montage composable : une disposition par format

Plutôt que cinq gabarits figés, l'outil utilise **un seul gabarit adaptatif**. Pour chaque format, le code choisit une **disposition** en fonction des proportions et du contenu, puis met le tout à l'échelle :

- **Bandeau de page catégorie** (site) : image d'ambiance de la catégorie en fond + **carte bleue** en surimpression (titre + offre).
- **En-tête d'accueil** (site) : mise en page **typographique** (accroche + offre/paliers + bouton), **sans image produit**, répartie sur toute la surface.
- **Avec image** (pubs / site illustrés) : le **produit détourné** est placé *dans le cadre*, **à côté** du texte (format large) ou **en dessous** (format étroit/portrait), et garde une part de surface garantie.
- **Typographique** (sans image) : accroche + offre/paliers + bouton, **répartis** pour remplir l'espace.
- **Formats fins** (728x90, 320x50) : tout passe sur une **ligne** et est **agrandi** pour remplir la bande.

Dans tous les cas, le contenu est **mesuré** après rendu et le cadre est **réduit ou recentré** pour tenir exactement : rien ne débordé jamais, quel que soit le brief.

10.2. La banque d'images

Les visuels produit sont organisés par catégorie, dans `assets/produit/categorie/<Categorie>/`, avec deux sous-dossiers : **Bandeau page categorie** (photo d'ambiance, pour le bandeau) et **promo** (produit **détourné**, fond transparent, pour les pubs). La **catégorie** est déduite de l'offre par Claude ; le code va chercher la bonne image. Pour une opération **transversale** (tout le catalogue), un dossier dédié `assets/produit/transverses/` fournit un fond de bandeau, une **promo par défaut** et un sous-dossier par **thème** (ex. Black Friday) avec son visuel. Claude reçoit la liste des thèmes et choisit, d'après le message, le visuel adapté (sinon la promo par défaut).

10.3. Une offre générique

L'outil sert des opérations commerciales variées : l'offre peut être un **montant** (« Jusqu'à 500 € »), un **pourcentage** (« -26 % ») ou un **avantage** en toutes lettres (« Livraison offerte »). Claude

renseigne **offre_type** et le gabarit rend chacun avec le **même style** (le bloc jaune de la charte qui épouse le texte), quel que soit le type. Pour les pourcentages, le signe « - » est ajouté selon le **sens** du message (présent pour « Jusqu'à 20 % », absent pour « Économisez 20 % »). Quand la mécanique comporte des **paliers** (ex. « 100 € dès 1000 €, 350 € dès 3000 € »), ils sont affichés en **pavés jaunes** (côte à côte ou empilés selon le format) sur les bannières qui ont la place ; sur les très petits formats, on retombe sur l'offre résumée. La **date de validité**, si elle est fournie, est affichée en mention complète.

11. Parcours utilisateur (UX)

La figure 4 illustre la boucle aperçu et ajustement, et la figure 5 un schéma d'écran (wireframe).

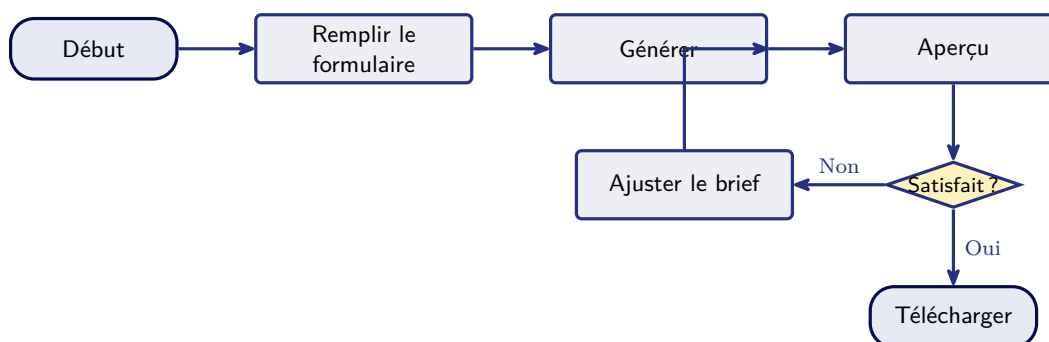


FIGURE 4 – Parcours utilisateur : la boucle aperçu, ajustement, régénération.

Générateur de kits graphiques (Gefradis)

1. Le brief

Mécanique de remise

Message et opération

Style (optionnel)

2. Les formats

☐ Site

☐ Google Display

☐ Google Pmax

☐ Meta

Générer le kit

Aperçu des bannières

Télécharger .zip

FIGURE 5 – Wireframe de l'interface (formulaire à gauche, aperçu à droite).

11.1. Style de l'interface

L'interface applique la charte à *la manière d'un produit*, et non comme une bannière : une **base neutre et claire** (fonds blancs et gris clairs de la charte, #F9FAFB et #F3F4F6) et la marque utilisée en **accents** (bandeau supérieur et boutons en bleu #253081, bouton d'action principal « Générer » en jaune #FFD52B, titres en police Cabin). L'objectif est un rendu d'outil professionnel et lisible, distinct du style « bannière » réservé au livrable.

12. Rôle de l'IA et stratégie de prompt

Ce que fait l'IA (Claude) : elle joue le rôle de *concepteur-rédacteur*. Elle rédige le copy (accroche, offre, paliers, date) et **déduit** quelques paramètres structurés (catégorie, type d'offre, thème transversal). **Ce qu'elle ne fait pas :** choisir la mise en page (la **disposition** est calculée par le code à partir du format), choisir la couleur de fond, l'image ou le bouton (ce sont des champs du **formulaire**), générer du HTML ou une image matricielle. Sa sortie est **uniquement un JSON** ; c'est le code Python qui l'injecte dans le gabarit **composable**, rendu en PNG par Playwright. Cela garantit l'absence d'erreur de parsing, des dimensions toujours exactes et le respect strict de la charte.

Stratégie de prompt : un *prompt système* fixe le rôle et surtout les **contraintes** (« Tu es le concepteur-rédacteur de Gefradis. Garde le même message sur tous les formats. N'invente aucun chiffre. Réponds uniquement via l'outil, en JSON, sans HTML. »). Le *message utilisateur* fournit le brief et la liste des thèmes transverses disponibles. Le **tool use** (schéma d'outil imposé) garantit un JSON valide. Un exemple figure en annexe (section 18).

Choix : pas de génération d'image au moment du rendu. La charte demande des photos *réelles* de produits, brutes et épurées. Les visuels proviennent donc de la **banque d'assets** de Gefradis (photos d'ambiance, produits détournés, visuels d'événements transverses fournis), placés par le code, plutôt que d'une génération à la volée. L'architecture reste prête à accueillir un module de génération si le client le souhaite (par exemple pour des fonds d'ambiance).

13. Plan de construction (lotissement)

La construction se fait par couches, de la plus simple à la plus robuste (figure 6) : chaque couche fonctionne avant d'attaquer la suivante, ce qui réduit le risque sur le planning.

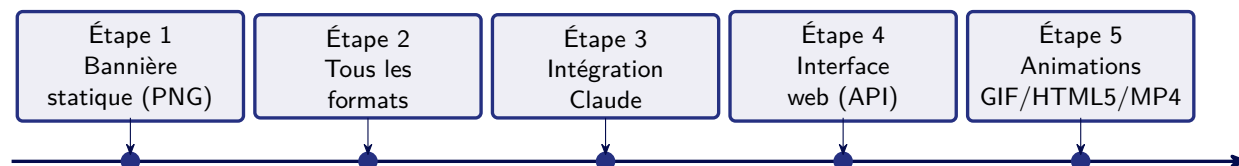


FIGURE 6 – Lotissement : étapes de construction successives.

Livrable cible : une **application complète**, qui fonctionne de bout en bout (toutes les étapes, animations comprises). C'est l'objectif, et non un périmètre réduit. Le lotissement ci-dessus est l'**ordre de construction** : on intègre le **flux complet** tôt (une tranche verticale sur un format), puis on enrichit en largeur (tous les formats), en mouvement (animations) et en finition (déploiement Cloud Run). Le module *photo produit* s'ajoute selon la réponse du client. Pour les animations, on privilégie le **GIF** (assemblé en Python avec Pillow à partir de quelques captures PNG) ; les formats **HTML5** et **MP4**, plus lourds, sont relégués en évolution (V2) pour ne pas mettre le délai en péril.

14. Déploiement et mise en production (GCP)

Le déploiement est une **phase à part entière** du développement. L'application est **déployée** sur Google Cloud, accessible à une URL publique (figure 7).

14.1. Conteneurisation

L'application est empaquetée dans une **image Docker** (Python, Playwright/Chromium et la police Cabin embarquée). La **même image** tourne à l'identique en local et en production, ce qui supprime les écarts d'environnement. Elle est stockée dans **Artifact Registry**.

14.2. Architecture en production

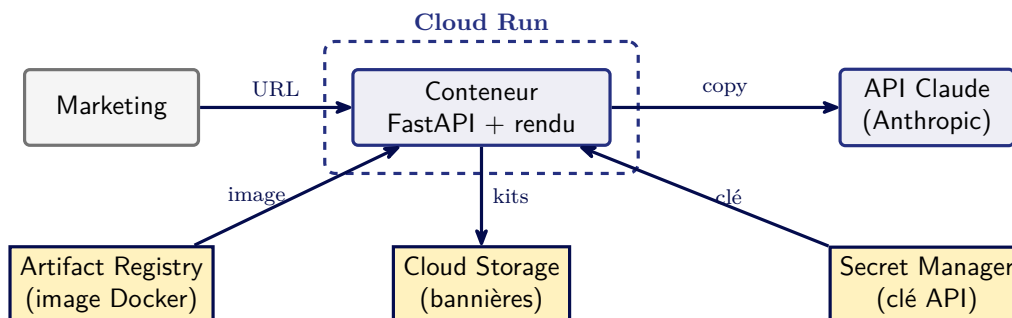


FIGURE 7 – Architecture déployée en production sur Google Cloud Platform.

- **Cloud Run** exécute des conteneurs à partir de l'image ; il scale automatiquement (plusieurs instances en charge, **zéro** au repos, donc 0 € inactif).
- **Secret Manager** fournit la clé de l'API Claude : jamais dans l'image ni dans le code, injectée à l'exécution.
- **Cloud Storage** stocke les bannières générées, dans un bucket partagé monté comme dossier de sortie, donc visible par **toutes** les instances.
- Le texte est produit via l'**API Anthropic directe** ; passer par **Vertex AI** (Claude dans Model Garden) serait l'évolution 100 % GCP-native (facturation et authentification unifiées).

14.3. Deux points d'attention résolus

- **Format de l'image** : les *attestations* ajoutées par défaut par BuildKit produisent une « manifest list » que Cloud Run refuse à l'import. On construit donc une image simple (`--provenance=false`).
- **Filesystem éphémère** : le disque de Cloud Run est **par-instance** et temporaire ; sans stockage partagé, des images générées manquaient d'une instance à l'autre. D'où l'écriture des sorties dans Cloud Storage.

15. Décisions, hypothèses et question ouverte

15.1. Décisions assumées

Sujet	Décision
Format de sortie	Images finales non éditables ; l'ajustement se fait en régénérant dans l'outil
Format statique	PNG
Livraison du kit	Archive .zip
Bannières animées	Plusieurs formats (HTML5, GIF, MP4) à partir d'une source unique
Google Pmax	Tailles standard
Mécanique commerciale	Donnée saisie par l'utilisateur : l'outil est générique
Référence visuelle	Fond bleu, titre blanc et chiffre clé dans un bloc jaune
Interface	Architecture découplée : backend FastAPI et frontend HTML/Tailwind ; UX en base neutre avec accents de marque
Fond, image, bouton	Choisis par l'utilisateur dans le formulaire (menus et champs), pas par l'IA : plus prévisible
Logo	Posé sur les bannières publicitaires uniquement ; variante selon la couleur du fond

15.2. Points clarifiés avec le client

- **Visuels** : le client fournit une **banque d'images** par catégorie (issue du flux Google Shopping) et des visuels pour les opérations transverses ; l'outil les place, sans génération.
- **Dimensions du site** : communiquées (HP 1900x456 et 426x575 ; catégorie 2560x280, sans version mobile).
- **Logo** : sur les bannières display (Google, Meta), pas sur le site.
- **Bouton d'action** : saisi dans le **formulaire** (champ dédié, optionnel) ; le principe du bouton a été validé par le client.

16. Risques et limites

Risque	Mitigation
Complexité des bannières animées	Statique d'abord ; pour l'animé, GIF assemblé avec Pillow ; HTML5 et MP4 en V2
Compatibilité Playwright et Python récent	Solution de repli (Pillow ou un rendu HTML alternatif) si besoin
Mémoire de Playwright (Chromium) en production	Allouer 2 à 4 Go de RAM sur Cloud Run ; limiter la concurrence des rendus
Logos fournis avec fond blanc (non transparents)	Détourage automatique mis en cache pour les poser proprement sur tout fond
Image presque carrée sur un bandeau très large (9 :1)	Image posée à pleine hauteur et répétée pour remplir, sans rogner le sujet
Coût et qualité du copy IA	Prompt contraint au format JSON, sorties courtes, modèle adapté au besoin
Complexité du frontend (FastAPI et Tailwind)	Frontend volontairement léger (pas de SPA lourde) ; construit progressivement

17. Critères de réussite et tests

- **C1** : le rendu respecte la charte (couleurs, Cabin, logo), vérifié par contrôle visuel.
- **C2** : chaque format demandé est produit aux dimensions exactes.

- **C3** : le copy généré est cohérent avec le brief (cas de test Gefradis).
- **C4** : la boucle aperçu, ajustement et régénération fonctionne.
- **C5** : le kit se télécharge en .zip et s'ouvre correctement.
- **C6 (démonstration)** : l'outil produit un kit complet et conforme à la charte sur le cas de test Gefradis.

18. Annexe : exemple de prompt

Prompt système (fixe, définit le rôle et les contraintes) :

```
Tu es le concepteur-redacteur de Gefradis (menuiserie PVC sur-mesure).
Regles STRICTES :
- Ton : professeur/mentor, precis et clair ; pas de superlatifs creux.
- Garde le MEME message (court) sur tous les formats.
- N'invente aucun chiffre : utilise seulement les conditions du brief.
- Reponds UNIQUEMENT via l'outil (JSON valide), sans HTML.
Champs : categorie, titre, offre_type (montant/pourcentage/texte), offre_label,
offre_nombre, offre_suffixe, offre_texte, paliers, date_validite, theme_transverse.
L'image, le bouton et la couleur de fond ne sont PAS choisis par l'IA :
ils viennent du formulaire.
```

Message utilisateur (le brief et le format) :

```
Operation : "Les jours Gros projets = Grosses Remises"
Mecanique : 500 EUR offerts des 1000 EUR d'achat HT
Format vise : Meta 1080x1080 (carre)
```

Sortie attendue (JSON seul, sans HTML) :

```
{
  "categorie": "transversale",
  "titre": "Gros projets = Grosses Remises",
  "offre_type": "montant",
  "offre_label": "Jusqu'a",
  "offre_nombre": "500",
  "offre_suffixe": "EUR offerts",
  "offre_texte": "",
  "paliers": [],
  "date_validite": "",
  "theme_transverse": ""
}
```

*Fin du cahier de conception (version 0.6). Version 0.6 : ajout d'une **section indépendante** « **Déploiement et mise en production** » (conteneurisation Docker, Cloud Run, Artifact Registry, Secret Manager, Cloud Storage ; API Anthropic directe, Vertex AI en évolution) reflétant l'application réellement **déployée en ligne**. Version 0.5 : gabarit **composable unique** (une disposition calculée par format, le contenu occupe tout l'espace et est mis à l'échelle, sans débordement) ; image, bouton et couleur de fond choisis au **formulaire** (et non par l'IA) ; gestion des **paliers** de remise et de la **date de validité** ; en-tête d'accueil typographique, sans image produit ; suppression de la pastille de coin (la remise est intégrée à la composition). Pour mémoire (0.4) : dimensions du site confirmées ; logo sur les pubs selon le fond (détournage automatique) ; banque d'images transverses par thème.*